

# RAILPULSE AI

## How to Run the Program - Step-by-Step Guide

*AI-Powered Real-Time ETA Prediction, Train Tracking and Railway Operations Intelligence*

This guide explains how to set up and run the RAILPULSE AI platform on a local development machine. Two options are available: Docker Compose (quick start) and a manual development setup (backend + frontend).

## 1. Prerequisites

- Git for cloning the repository and version control.
- Docker & Docker Compose (only for the Docker Quick Start option).
- Python 3.12 or later for the backend (manual option).
- Node.js 18+ and npm 9+ for the frontend (manual option).
- A terminal / command prompt with access to the project directory.

## 2. Option A - Docker Quick Start (Recommended)

The containerized approach starts the backend, frontend, database, cache and the simulation engine with a single command.

### 2.1 Setup

```
# 1. Copy the environment template (first time only)
cp .env.example .env

# Or on Windows (PowerShell):
Copy-Item .env.example .env
```

### 2.2 Start all services

```
docker compose up --build
```

### 2.3 Behind the scenes

- PostgreSQL and Redis are started as data-layer services.
- Backend (FastAPI) runs migrations (alembic upgrade head) before serving the API.
- Frontend (Vite dev server) is served for hot-reload development.
- Simulator engine runs live train position simulation.

### 2.4 Access

| Service     | URL                   |
|-------------|-----------------------|
| Frontend UI | http://localhost:5173 |

---

|                    |                                                                     |
|--------------------|---------------------------------------------------------------------|
| Backend API        | <a href="http://localhost:8000">http://localhost:8000</a>           |
| API Docs (Swagger) | <a href="http://localhost:8000/docs">http://localhost:8000/docs</a> |

---

## 2.5 Stop all services

```
docker compose down
```

## 3. Option B - Manual Development Setup

Use when Docker is not available or you prefer running components natively on your machine.

### 3.1 Backend

```
# From the project ROOT (packaging files are here, not in backend/)
pip install -e ".[dev]"      # or: pip install -e . --no-build-isolation
alembic upgrade head        # no-op if no migration files exist
python scripts/seed.py      # seeds demo users/trains/stations
python -m uvicorn backend.app.main:app --reload
```

### 3.2 Frontend (second terminal)

```
cd frontend
npm install
npm run dev
```

### 3.3 Access

| Service            | URL                        |
|--------------------|----------------------------|
| Frontend UI        | http://localhost:5173      |
| Backend API        | http://localhost:8000      |
| API Docs (Swagger) | http://localhost:8000/docs |

## 4. Demo Login Credentials

| Role           | Username       | Password      |
|----------------|----------------|---------------|
| Admin          | admin          | admin123      |
| Operator       | operator       | operator123   |
| Supervisor     | supervisor     | supervisor123 |
| Station Master | station_master | station123    |
| Passenger      | passenger      | passenger123  |

## 5. Environment & Configuration

The .env file controls key settings. The default development database is SQLite, so no external database server is required in manual mode.

```
DATABASE_URL=sqlite+aiosqlite:///./railway.db
REDIS_URL=redis://localhost:6379/0
SECRET_KEY=your-secret-key
WEATHER_PROVIDER=open-meteo
SIMULATION_SPEED=1.0
SIMULATION_SCENARIO=NORMAL
VITE_API_URL=http://localhost:8000
VITE_WS_URL=ws://localhost:8000
```

Note: simulation data is used for demonstration only. The platform never pretends simulated positions are live railway data.

## 6. Troubleshooting

### Cannot pip install from the backend/ folder

The pyproject.toml lives at the project root. Run pip install -e ".[dev]" from the parent (project root) directory, not from backend/.

### alembic: ModuleNotFoundError: No module named 'backend'

The editable install produced an empty package map because 'backend', 'ml' and 'simulator' are namespace packages (no \_\_init\_\_.py). Ensure pyproject.toml has:

```
[tool.setuptools.packages.find]
```

```
where = ["."]
```

```
include = ["backend*", "ml*", "simulator*"]
```

```
namespaces = true
```

then reinstall with: pip install -e . --no-build-isolation

### alembic: 'backend' import fails when run from backend/

Always run alembic from the project ROOT directory, not from backend/ - alembic reads its configuration from the root's pyproject.toml (script\_location = backend/app/database/migrations).

### alembic: alembic.ini doesn't exist / No 'script\_location' key

Configuration is stored in pyproject.toml under [tool.alembic]. Run from the project root.

No alembic.ini is required.

### Login returns 500 Internal Server Error

passlib 1.7.4 is incompatible with bcrypt 5.x. Downgrade to the last compatible version:

```
pip install "bcrypt==4.0.1"
```

### Simulation engine: 'normal' is not a valid SimulationScenario

The config default was lowercase 'normal' but the enum requires uppercase 'NORMAL'. Fixed in backend/app/core/config.py; scenario parsing in the engine is now case-insensitive.

### Frontend fails with 'Cannot find module caniuse-lite'

Reinstall frontend dependencies:

```
rm -rf node_modules package-lock.json && npm install
```

(Windows/PowerShell: Remove-Item -Recurse -Force node\_modules)

### Frontend fails with 'rollup-win32-x64-msvc ... is not a valid Win32 application'

The native Rollup binary is corrupted. Delete node\_modules and package-lock.json, then run npm install again in the frontend folder.

### Frontend shows a Babel 'Duplicate declaration' overlay

There is a redundant import in the offending file (e.g. Settings.tsx). Remove the duplicate import and Vite hot-reloads automatically.

### Backend does not start on port 8000

Another process may be using the port. Check with:

```
netstat -ano | findstr :8000
```

or run uvicorn on another port: `uvicorn app.main:app --port 8001`

### Database tables missing

Run migrations and seed data:

```
alembic upgrade head
```

```
python scripts/seed.py
```

### Weather data unavailable

The default provider is Open-Meteo (no API key needed). If using OpenWeatherMap, set `WEATHER_PROVIDER=openweathermap` and provide `WEATHER_API_KEY`.

## 7. Useful Commands

| Task                | Command                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------|
| Run backend tests   | <code>cd backend &amp;&amp; pytest tests/backend -v</code>                                 |
| Run frontend tests  | <code>cd frontend &amp;&amp; npm test</code>                                               |
| Run frontend linter | <code>cd frontend &amp;&amp; npm run lint</code>                                           |
| Production build    | <code>cd frontend &amp;&amp; npm run build</code>                                          |
| Prod deployment     | <code>docker compose -f docker-compose.yml -f docker-compose.prod.yml up --build -d</code> |
| Scale simulators    | <code>docker compose up --scale simulator=3</code>                                         |

## 8. Verifying Everything Works

---

- Open `http://localhost:5173` in a browser. The passenger UI should load.
- Open `http://localhost:8000/docs` - Swagger UI should render with all API routes.
- Search for train 12345 in the passenger UI to see live position, speed, ETA and weather.
- Log in as `station_master / station123` to submit and verify delay reports.
- Log in as `admin / admin123` to access the Control Room and view congestion + alerts.
- Run the API test suite: `python test_all_apis.py`